

TOURMALINE CODE

Engineering magazine
about code, processes,
and business solutions

test-driven development

{TESTS FIRST,
CODE SECOND}

```
def test_premium_user_gets_10_percent_off_order_total():
    order = order_with(
        items=[
            ("book", 500),
            ("book", 500),
            ("magazine Tourmaline Code", 1),

            ("TDD extreme programming", 10),
            ("Code without bugs", 10),
            ("Code sustainability", 1000),
        ],
        user=premium_user(),
    )

    user_name = order.get_username()
    user = UserCommand.get_by_username(user_name)
    assert user.type == "premium"

    total = order.total()
    assert total == 900
```

{IN THIS ISSUE}

TDD ACROSS
DIFFERENT AREAS
OF DEVELOPMENT

WHEN TDD
ACTUALLY
WORKS

№1

09.2026

Test-Driven Development

Hi,
We are Tourmaline Core.

For the past seven years, we've been developing IT projects for a wide range of companies and organizations.

Carl Zeiss and the Chelyabinsk Zoo, medical start-ups and an Australian charity. Drivers, ML-models, web systems, firmware, blockchain, Content Credentials...

Over the years, we've collected plenty of stories: about the decisions we made, the results they brought, the mistakes we learned from, and the lessons that changed how we approach subsequent projects.

All of that, bit by bit, laid the tracks that we put our new projects on. Some of our stories become articles on Habr.com, others turn into talks at conferences. But some pieces of the puzzle don't fit these formats: those stories have tons of characters, tons of references, and every subplot drags a dozen others along with it. These are the kind of stories that will make up this magazine.

The first issue is about Test-Driven Development. The topic wasn't chosen at random: several years ago, our CEO made a fundamental decision — from then on, we would stick to TDD whenever it is possible. Today, it is the foundation of our engineering practice.

And on every project we explain this choice to technical specialists and managers on the client side. We've decided it's time to collect our responses to both groups in one place.

Maria,
Chief Development Officer
of Tourmaline Core



What is TDD?

In 2001, Hacker magazine published a legendary article called “If Programmers Built Houses”. The construction crew forgot to lay the foundation before building the ground floor, so the whole thing had to be demolished. Then on the fifth floor a wall turned out to be at a 40-degree angle. Then a brick was laid wrong and it was impossible to tell which one, so it was easier to just tear the house down and rebuild it. The roof was built separately and didn’t fit the house. After optimization, the elevator broke through the roof and flew away.

Of course, when it comes to building a house in real life, everything is planned out in advance. Everything needs to be agreed on: the number of floors, the load they’ll bear, the placement of load-bearing walls, the type and material of the exterior cladding. The technical specification defines how the house is supposed to turn out.

When it comes to software development, it’s often the other way around. A programmer writes code and only then writes automated tests for what has already been done. And even that’s not always the case. These tests validate the decisions that were made during the development process — they don’t help design those solutions.

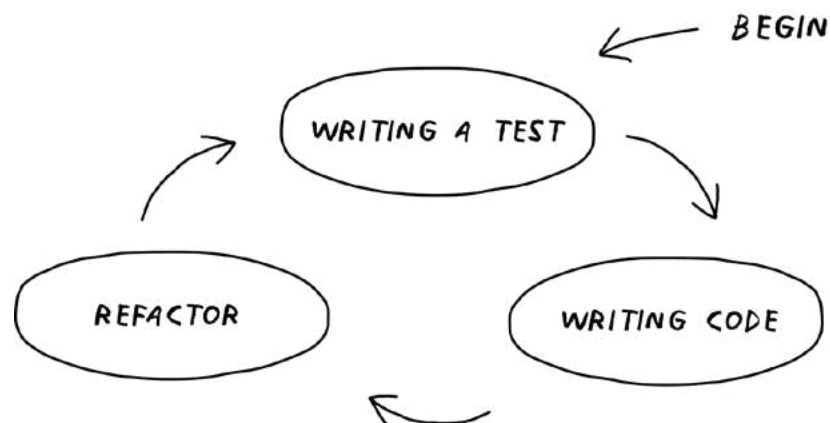
That’s why IT systems so often resemble the house from that Hacker tale.

TDD (Test-Driven Development) is a development approach where you agree on what to build first and implement second.

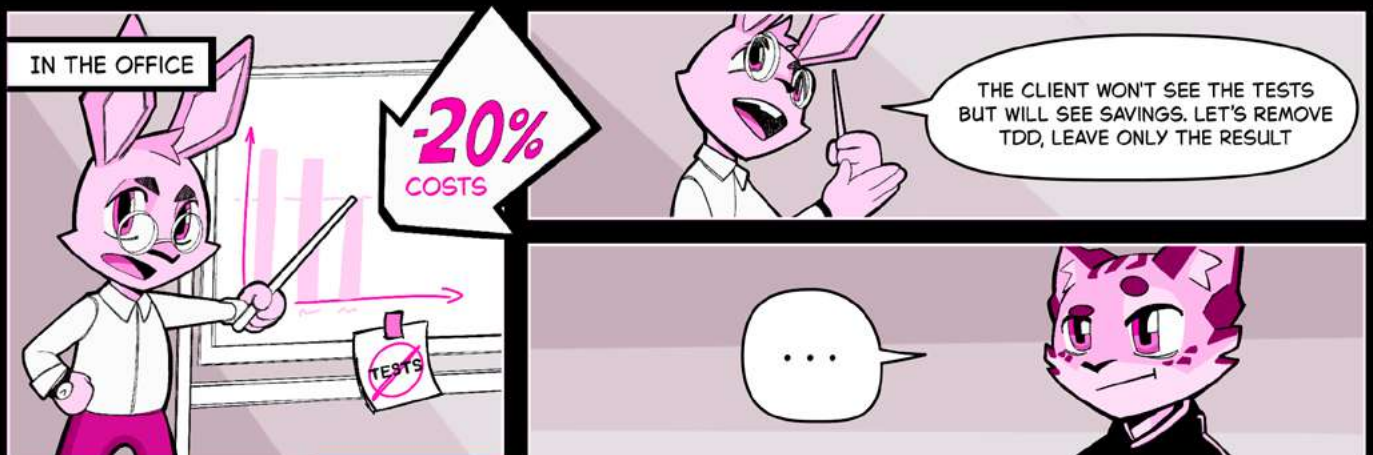
A programmer writes down the expected behaviour of a new function: what its inputs are, what it is supposed to return, what is considered an error. This is called a test, it will validate the code automatically. Then comes the actual implementation. After that, the code gets cleaned up and perfected without changing its functionality — that’s called refactoring. The cycle is repeated dozens of times a day, in small steps.

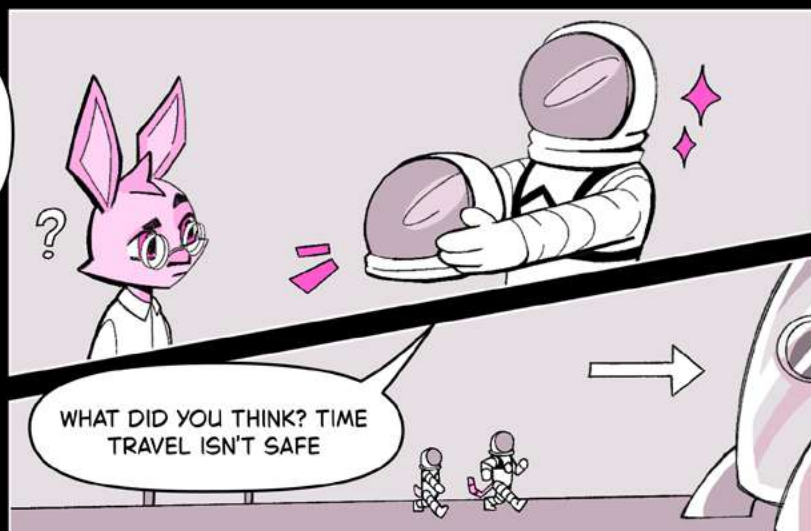


IF PROGRAMMERS
BUILT HOUSES



**So why did we choose this path: developing with TDD?
We'll try to explain it with a comic strip.**





A MONTH PASSES,
ON A CALL WITH THE CLIENT

IT'S GREAT THAT
YOU FINISHED
IT ON TIME!

WE'LL PASS IT TO OUR QA
DEPARTMENT FOR TESTING

SEE!
IT'S ALL GOOD

JUST
WAIT...

A MONTH PASSES

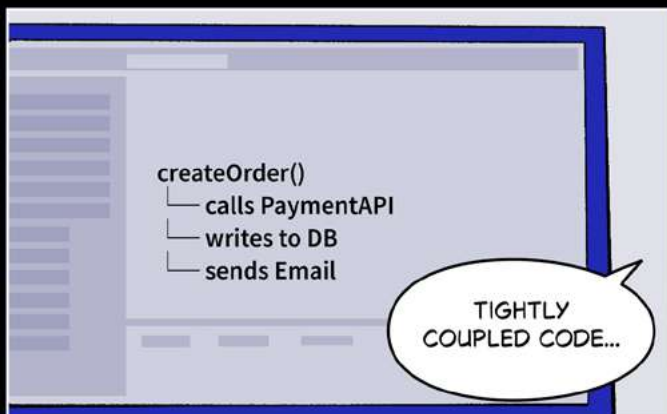
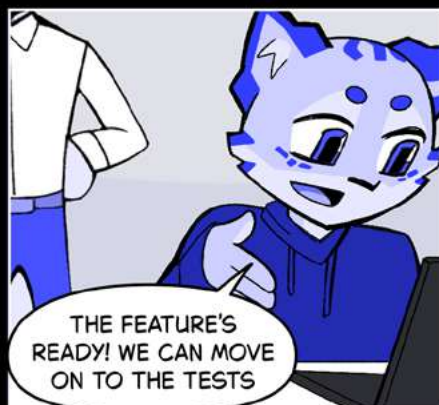
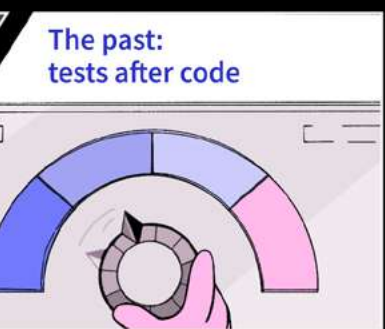
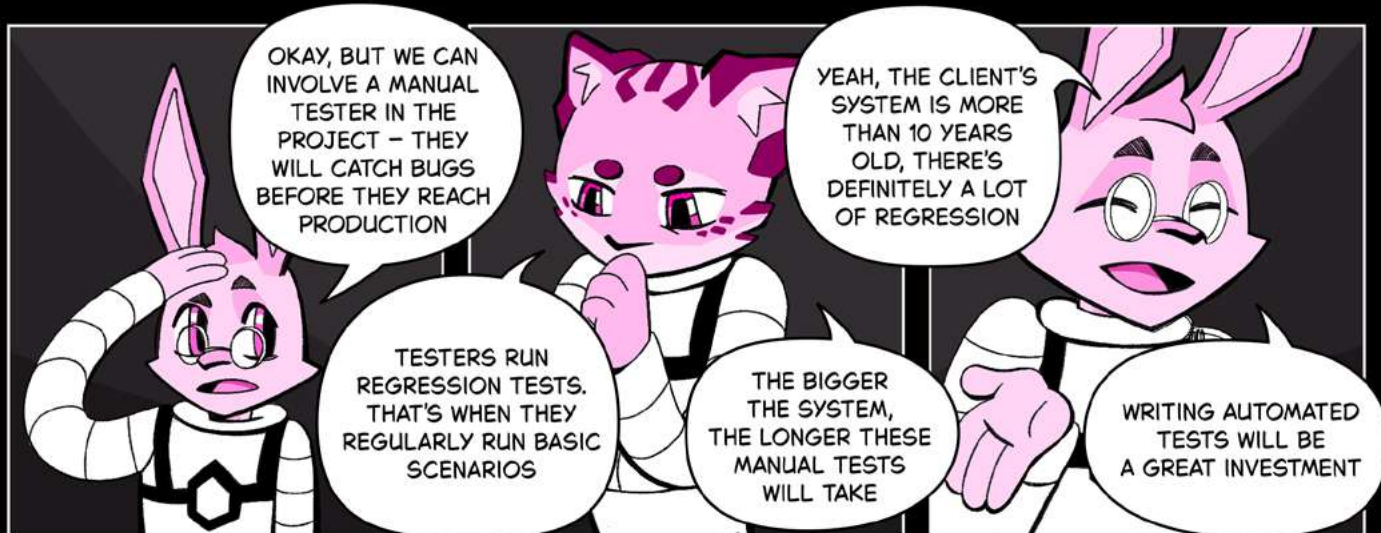
CLIENT:
THEY DETECTED ANOTHER BUG!

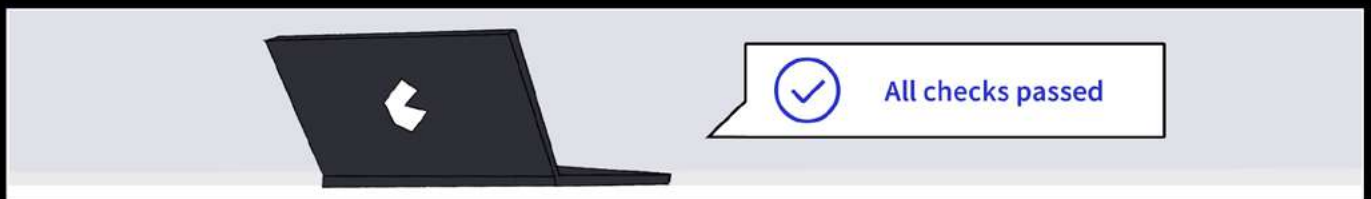
CLIENT:
YOU HAVEN'T FIXED THE PREVIOUS ONES YET!

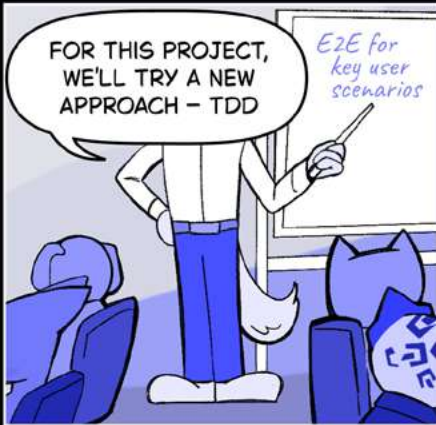
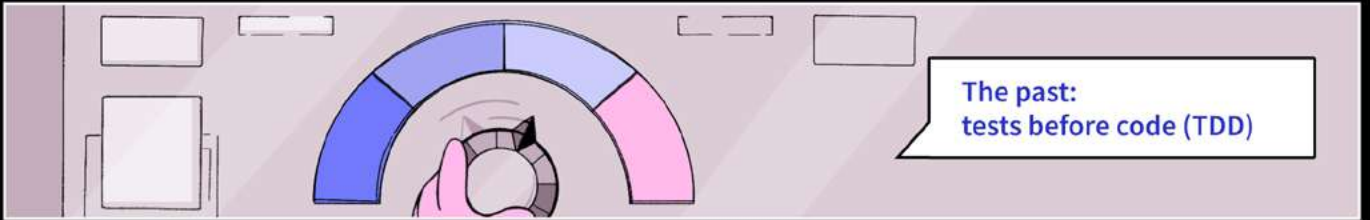
CLIENT:
HOW COULD YOU MESS UP SO BAD?!

WE SPENT ANOTHER
MONTH FIXING IT ALL.
DIDN'T FINISH IT IN TIME
FOR THE RELEASE

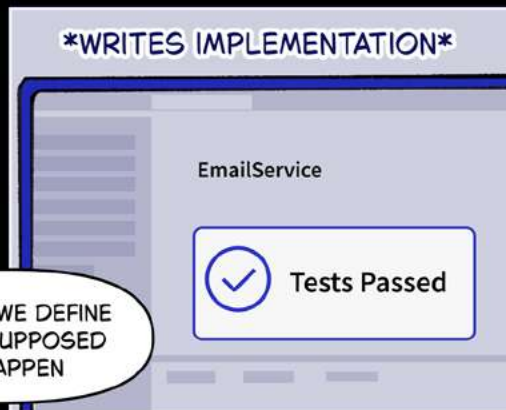
AND WE LOST THE CLIENT -
THEY WON'T TRUST US
WITH ANOTHER PROJECT



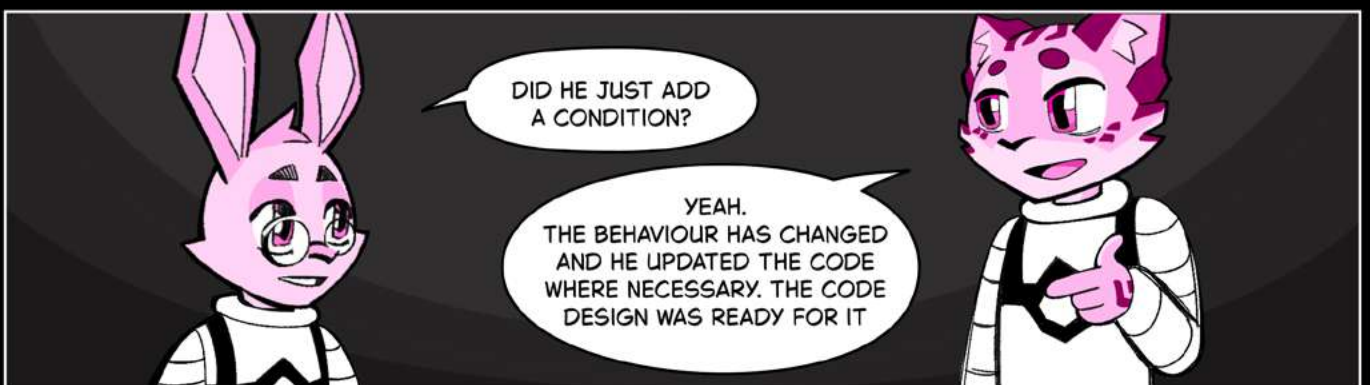




AT FIRST WE DEFINE
WHAT'S SUPPOSED
TO HAPPEN



SEE? HE DEFINES
THE BOUNDARIES
BETWEEN PARTS OF
THE SYSTEM RIGHT
AWAY. THE TEST
MAKES HIM THINK
ABOUT CODE DESIGN
FROM THE BEGINNING,
NOT AT SOME
LATER POINT





THERE ARE NO UNIVERSAL SOLUTIONS – ONLY SOLUTIONS SUITABLE FOR THE TASK AT HAND

IT ALL DEPENDS ON THE BUSINESS OBJECTIVES AND THE SYSTEM THE DEVELOPER IS WORKING WITH



Approach	What we get	When it makes sense
No tests	Quick at first, expensive later	MVP. Prototype. R&D experiment. Any code that you're ready to throw out and start from scratch.
Tests after code	You get a certain level of control, but changes are expensive	A system with a lifespan of 1–1.5 years. Stable requirements.
TDD	Expensive at first, cheaper in the long run	Systems with a lifespan of 2–3 years. Rapidly changing requirements. High cost of a bug in production.

We don't include manual testing as a separate, full-fledged strategy in this table: it can't be scaled to projects with long lifespans and frequent changes, and manual tests are most of the time replaced with automated tests or selectively complement them.

This Isn't Just Our Experience

When we suggest TDD to a client, the reaction is predictable: sounds nice, of course, but isn't it more time-consuming and expensive? Besides, how do you assess the impact? Who says it will give better results?

In 2008, Microsoft Research, IBM, and the University of Northern Carolina published their research results regarding four production teams. Three of them were Microsoft's: the teams that develop Windows, MSN, and Visual Studio. And the fourth team was from IBM, they developed drivers for devices.

Totally different teams and products: different amounts of code (from 6 to 155 thousand lines of code), different team sizes and project durations (from 24 to 119 man-months), different domains.

These teams used TDD. Alongside them, there were other teams working on similar projects not using the TDD approach.

And here's the result:

TDD teams' defect density turned out to be

40–90%

lower than that of comparable teams that didn't use TDD.

The former had to pay for it with time: the managers estimated the increase in initial development time at

15–35%

An important detail: the estimate covered the initial development stage only. It doesn't include further product maintenance. But that's exactly where most of the time goes: bug fixes, regression testing after every change, incident discussions, immersing new team members into the codebase.

TDD teams have faster maintenance processes since the code has fewer defects, tests describe its behaviour, and the code covered by automated tests can be changed without fear: the tests will show if anything breaks.

In other words, **15–35%** of the time and costs is a price you need to pay once in exchange for the quality that will last longer.

The cost-effectiveness of the TDD approach can be gauged by the life expectancy of the project itself: if the project is going to live for a long time, the investment will pay off.



RESEARCH
BY MICROSOFT
AND IBM

Contrary Evidence

In 2021, a meta-analysis of 12 experiments was published. Both beginner and experienced programmers took part in the experiments, but none of them had used TDD before. The tasks were instructive and had a 2.5-hour time limit. In this research, TDD didn't show any advantage over iterative development with short cycles and tests written after the code.

This research doesn't contradict the findings of Microsoft and IBM but complements them with a new idea: in a session lasting only a few hours, people with no prior TDD experience don't have enough time to see the effect.

We often work on projects with longer lifespans, so we look to Microsoft and IBM as examples.



CONTRARY
EVIDENCE

Seven Conditions for TDD to Work

In 2011, researchers at Mälardalen University reviewed TDD-related studies from the industry and wondered why an approach that demonstrates such strong results in research catches on so poorly in practice. This publication usually gets cited as “seven reasons not to use TDD”.

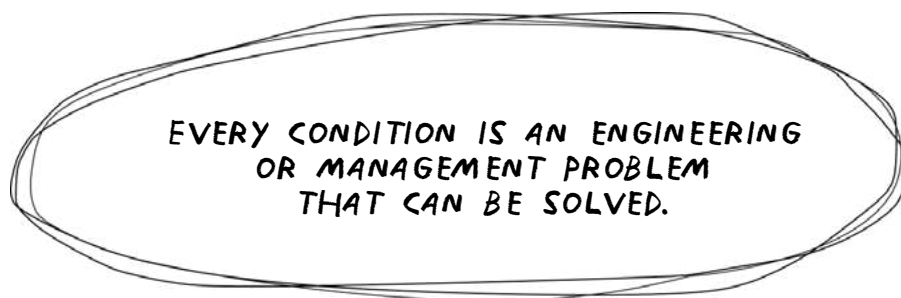
We reworked the article's conclusions into a checklist of seven conditions that allow TDD to produce that very effect documented by Microsoft and IBM — an effect that our team's experience confirms.

- 1** **Willingness to pay upfront.** The team, managers, and clients understand in advance that the tasks will take more time during the transition to TDD. If everyone involved isn't on board with this idea, TDD will soon get abandoned under the pressure of deadlines.
- 2** **Team's experience with TDD.** Oftentimes, the issues with adopting TDD stem from the lack of understanding of how this integration should happen. You can start filling these basic conceptual gaps with books by the founding father of TDD, Kent Beck, video tutorials, and open-source repositories.

- 3 **Test-writing skill.** Poorly written tests are fragile; they duplicate each other and validate the implementation instead of behaviour. With such tests, TDD turns into an expensive, unmaintainable illusion of control.
- 4 **Basic code design.** Even with TDD, where one of the functions of tests is to help shape the code design, it's necessary to have a sense of a skeleton of the module that's being developed: scopes, interfaces, responsibilities. Without that, tests get rewritten at every step.
- 5 **Discipline in sticking to the cycle.** TDD only works if a team truly sticks to this approach, rather than imitating it. In practice, discipline may suffer in this case because of the lack of time or unwillingness to follow the approach.
- 6 **Suitable domain and tooling.** The article's authors write about the lack of testing tools in certain areas, such as interface testing, and about the need for further development. The article dates back to 2011. Plenty of time has passed since then, and the industry has made a leap forward — web interfaces now have at least Cypress and Playwright. The testing industry has grown significantly: you can find tools and adapt them to the needed domain.
- 7 **No legacy.** TDD in its truest form doesn't work on legacy code that has no test coverage. First of all, the system requires a separate kind of effort to prepare it for the new approach, and only then TDD can be set into action. This is a whole other project that needs to be estimated separately.

The good news is that none of the conditions rest on magic or the talents of some "All-Star Team".

- Willingness to pay upfront is a matter of negotiating with the client.
- Experience, test-writing skill, and discipline in using TDD can be trained.
- Basic code design and appropriate tools are defined during estimation.
- Legacy code becomes a part of a separate workstream.



What Can TDD Be Compared to in the World Outside of IT?

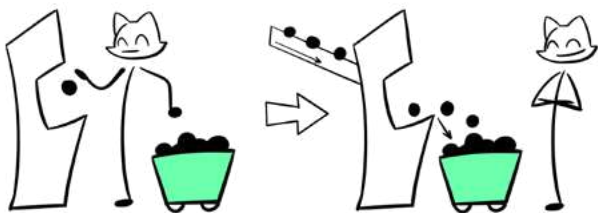
Toyota Production System (TPS) has given us not only amazing automobiles but also its vision of how to design a production process so that quality control would be an integral part of the process, not just a final step. There are several different concepts that the company uses for that goal, such as Lean (production with minimal waste), Kanban (visualizing the workflow), Kaizen (the philosophy of taking small steps).

In the TDD context, the most interesting principles of Toyota's vision are Jidoka and Ji Kotei Kanketsu.

Jidoka

Toyota describes the Jidoka principle as “automation with a human touch”. Its main components are automatic defect detection during production, immediate stoppage of the process once a problem is detected, and wise involvement of a human to help solve the issue.

Sakichi Toyoda started developing the principle long before integrating it into production. The future founder of Toyota Group was looking for ways to make his mother's work easier as she was weaving on a hand-operated machine late into the night. He invented an automated loom, whose main feature was not just replacing manual labor: the loom could stop itself if a thread broke. This feature of the loom helped boost both productivity and efficiency — and that's where the Jidoka idea came from.



In Toyota's automobile production, that idea transformed into automatic systems handling everything that a machine can be trusted to do, while a human handles everything that requires judgement. A machine notices a flaw and stops; then a human steps in, finds the root of the problem and makes a decision about what to do next.

Andon —

is a system that notifies workers of a quality problem. It starts with a cord that can be pulled to stop the entire production line. The system or any worker on that line can pull the cord upon noticing a problem, production stops, a panel lights up, and a team lead goes over to it. If the lead manages to solve the problem within an instant — the line continues; if not, the entire conveyor stops until the situation is resolved. Everyone has the right to stop production, not just team leads or QA engineers. The system itself can detect a defect and stop — but that doesn't eliminate the human element, only helps to reduce the human factor in production control processes.



At first, it may seem that the stoppage of the entire conveyor means immediate losses for the company: workers and machines are idle, production targets aren't being met. But in reality, a defect that makes it further down the conveyor has a higher cost: it has to be taken out of an assembled unit or a mechanism at later production stages, and if it gets as far as the dealer — it costs the company its reputation.

Ji Kotei Kanketsu (JKK)

The key idea: a defect needs to be detected and dealt with right where it occurs because the next stage of the work process is your customer, who must not receive a defective item.

Every worker must deal with a problem the moment it arises. An important component of this principle is the standardization of the work — a clear understanding and visualization of what every worker should do as part of the overall system. Without standardization, the next process never knows which part it will receive this time.



Now let's go back to development

And draw an analogy. A test written before the code is essentially the same as the broken-thread detector built into a loom or the defect detector on a production line. Developers who use TDD place such detectors all over their product throughout the project.

This is not a quality control department that gets a production batch at the end of the process and just shrugs; it's a mechanism that's built into production and knows how it's supposed to work. It doesn't tolerate a sudden change in the shape of a wheel or a driver's seat facing an opposite direction. A red test that failed is an andon. It doesn't just communicate that something's broken somewhere; it lights up a panel right above where the defect occurred.

Toyota's philosophical approach to work based on several important principles and automation goes beyond production halls. The focus on preventing defects at the earliest stage possible, root-cause analysis, standardization and expansion of the workers' abilities to intervene in the work processes of the entire company — all of this fosters a culture of taking responsibility for quality.



Anatomy of Test: Good vs. Bad

There is one particularly challenging prerequisite for TDD to work: the test-writing skills. It's better to start small: look at the examples of bad and good tests and figure out what makes them different.

```
def test_order():
    o = Order()
    o.add(Item("book", 500), 2)
    o.user = User(status="premium")
    o.recalculate_order_sum()

    assert o.total == 500 * 2 * (1 - PREMIUM_DISCOUNT)
    assert o.items_count == 2
    assert o.user.status == "premium"
    assert o.log[-1] == "recalculated"
```

Bad test

So what's wrong with this test?

- 1 A name that doesn't help to understand the purpose of the scenario and the assertion right away. "test_order" could be the name of a test folder's at best, not the name of a specific check.
- 2 The total is calculated using a formula that's most likely present in the exact same way in the implementation code. Then what are we checking?
- 3 One test has four assertions for four different things. A quality control system must notify us when something specific breaks and instantly show which part of the code fails. With this implementation, it would be unclear whether the sum calculation was incorrect, a user's status wasn't recorded, or a number of items in an order wasn't taken into account. This kind of check doesn't help identify where something went wrong or how to fix it.
- 4 We see the actual implementation in the test: we create an order, add an element to it, see the method used to recalculate the order's total cost. The code's encapsulation is broken. It becomes an implementation check, not a behavioural one. The smallest change in the logic of the implementation code or even refactoring would mean that these tests need to be rewritten as well.

```
def test_premium_user_gets_10_percent_off_order_total():
    order = order_with(
        items=[
            ("book", 500),
            ("book", 500)
        ],
        user=premium_user(),
    )
    total = order.total()
    assert total == 900
```

What, on the contrary, is this test getting right?

- 1 The name. It's transparent: we can understand what happens, who it happens to and when. Even if you mention this test's name to a manager at a daily meeting, they'll be able to figure out what's being checked.
- 2 No formulas mirroring the implementation code. If a total equals 900, it equals 900 — a defined number that's known in advance according to the business rules, not by some formula that can be wrong or completely repeats the implementation.
- 3 Atomicity. One test checks one aspect and doesn't try to cover everything. That doesn't mean that there must only be one assert expression. One test may include several assertions but they should all be related to the same behaviour. Three assert expressions describing one behaviour are normal. Three assertions describing different behaviours are not.
- 4 The test doesn't know anything about how the order's total is actually calculated. Order creation is hidden behind a factory. The implementation can be changed, refactored, but the test will remain green because it has only ever covered the business scenario. This test checks the behaviour, not the implementation.

A good test survives refactoring — a bad one has to be changed along with the implementation.

What are some other signals that something's not right?

- 1 The test fails every other time. Such tests are called flaky. Today it's green, tomorrow it turns red even though the code stays the same. Frequent offenders are race conditions, randomly generated data, time dependency, or an object shared between tests that was changed in one test and then used in another, with the second test expecting its previous value. Such a test is worse than no test at all since a team quickly learns to rerun tests waiting for a successful attempt instead of reacting to the failure, looking for the error and fixing it.
- 2 The test's success depends on its position in the overall test queue. That's a bad idea. Tests should not depend on each other. Then they won't be flaky and can also be run in parallel to speed up the test run.
- 3 The test implements some logic. If it has conditions, loops, error handling, or formulas, it is a good indication that the test itself needs to be tested. A check should be simple enough so that its correctness is apparent at first glance.
- 4 The test has never failed. If you "break" a piece of code and the test remains green, it means it never checked anything. That's why with TDD, it's important to run a test right after you write it to see that it fails. And that it passes only after the implementation has been added.
- 5 The test setup is ten times longer than the actual check. Forty lines of setup for a single assertion is often a sign that the code lacks the tools necessary for writing simple, high-quality tests.
- 6 You can't name the test. If you can't come up with a meaningful test name and end up with something like "test_process_data_2", that means the tested part doesn't have a defined behaviour or a purpose.
- 7 There's a comment above the test describing what it does. That means either the test has a bad name or it is hard to read. Neither says anything good about the test.

A bad name and excessive checks can easily be spotted with the naked eye. Other indications reveal themselves during a test's lifespan and maintenance cycle.

About Arrange-Act-Assert and Given-When-Then

All good tests look alike; every bad one is bad in its own way. One thing any good test definitely has is structure. Such tests can be divided into three parts: the setup of the state, the action, the validation of the result.



This structure has a name: in software testing, it's called Arrange-Act-Assert.

In BDD (Behavior-Driven Development), this structure is called Given-When-Then. Dan North and Chris Matts came up with the formula in 2004: it was needed to describe a system's behaviour with words familiar to everyone on the team.

The language for writing scenarios that turns this formula into text is called Gherkin.

A scenario written in it looks like this:

Scenario: Premium user gets a 10% order discount
Given user with the status "premium"
And order for 1000 galleons
When system calculates the total
Then total equals 900 galleons

Using Gherkin syntax to write tests expands the idea of "a good test's name can be mentioned at a daily meeting and the manager will understand it" to the fullest extent: by default, a test scenario is written in the same language the company uses to speak about the product.

This syntax may not be one-size-fits-all for writing tests for any product — it has its limitations. But what you can certainly borrow for your tests is the structure. And if this structure can't be easily identified in a test, if the setup is mixed with the validation, the validation is mixed with the setup, if there are multiple actions — that means the test should probably be reconsidered. A test with a clear structure paints a picture in words that doesn't need to be translated.

Testing Bookshelf



The examples of good and bad tests in this article are just a small part of the kung fu that usually takes years to master. To shorten this journey, we can turn to other testers' experience described in books.

1 *Extreme Programming. Test-Driven Development* by Kent Beck.

The core book on the topic by the author of the TDD concept. Contains analysis of real projects and a collection of tactics.

2 *Test design. Practical Guide for Beginners* by Olga Nazina.

Answers the question of what to check, not how to write it. Equivalence classes, boundary values, pairwise and pre-made checklists.

3 *Working Effectively with Legacy Code* by Michael Feathers.

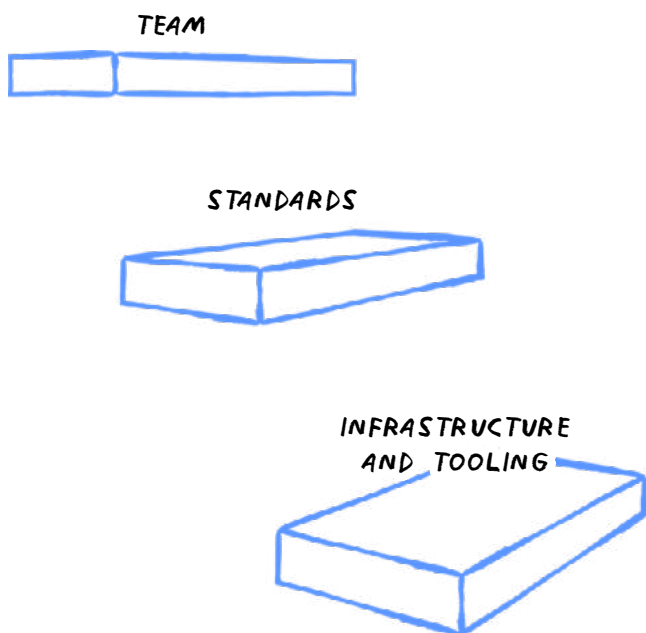
What to do when you can't start the project cleanly from scratch. Tricks that help you add tests to the code that was written without them.

Three Foundations of TDD

For the approach of writing tests before implementation to work, it's not enough to choose a suitable library, tell the developers to only work this way from now on and begin the next day. That simply can't work. Writing tests before code isn't some tool you can just add and forget about. It is a process that requires maintenance and control. If nothing's done to maintain it, in a couple of weeks the team will go back to writing tests after code if there's time left. And there's never enough time.

Moreover, the development process will slow down during the transition period. The decrease in speed is noticeable off the bat, while a leap in quality will take no less than a quarter to show.

To survive the transition period, spot the first results, and gain the energy to move forward, TDD must be supported on different levels:



First, picture the result. Then, start working towards it. That's what TDD is about, and it follows the exact same rule: first, you plan out the integration of this process, then you act on the plan.

Infrastructure and tooling level:

Everything needed for the tests to run must be in place. This sounds trivial, but the necessary setup isn't always there. Choose the frameworks for writing tests. Configure running tests in a local environment and in production. Keep the existing tests that pass consistently, the inconsistent ones should be fixed or removed. Adding tests should be easy; it shouldn't feel like torture. Running tests should also be quick and easy: if a test run takes dozens of minutes, the TDD cycle will become extremely long, the developer will have to switch context, and coming back to a new TDD cycle will take longer.

Standards level:

Standardization helps different team members to follow the same steps and get similar results. Like a recipe gives step-by-step instructions on making a pie, testing strategy defines which tests should be written during development and at what level. Not all cases will fit into a standard, but that's not really necessary: testing is an art and a craft, and not all cases can be tested the same way. However, basic create-read-update-delete scenarios are expected to look the same in a project regardless of the author: the same structure, the same naming rules, and the same places to look for tests.

Team level:

The team must recognize the value of the approach. If believing that everything will be fine doesn't work, it's enough to have the flexibility to try something new as an experiment: on a single module, with a fixed deadline and predefined success criteria. A leader keeps an eye on discipline, participants follow the approach without sabotaging it. Though the sabotage rarely ever has malicious intent behind it — more often, it is an indication that in some places, it's too difficult to write a test or unclear how to do it. In this case, either the infrastructure needs to be fixed, or the testing strategy needs an additional planning session with the team and a QA engineer.

Yeah, we could. But what for?

Bingo of objections to writing tests before implementation.



Twenty phrases but only six different arguments.

Find which one resonates with you the most — the options are colour-coded:

“Not now”

Don't measure time to commit. Try to measure time to a release, including all the updates after reviews, bug fixes, and debugging someone else's code a month later. You'll get a different estimate.

“Unnecessary in this case”

It's not the amount of code that matters, it's the code's lifespan. The longer it lives, the more often you need to check that a change in one part doesn't break the rest.

“Can't be tested”

Can't test today — scared to apply changes tomorrow. That's tech debt that needs to be dealt with, not ignored.

“Makes no difference”

A test written after the implementation is adapted to the existing code. You've never seen it red — are you sure it can fail at all?

“Someone else will test it”

Everyone looks at the code and wonders whether it does what it's supposed to. What it actually does — only its author knows (if they remember).

“I can't do it”

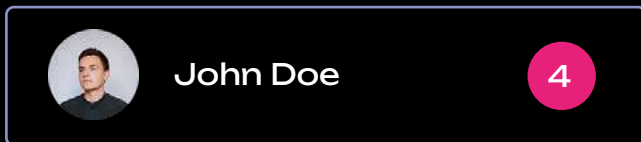
If it's hard to describe, the task is still unclear. Don't know what the UI will look like — a test will help to come up with it. A test doesn't interfere with your thinking, it is a way to think.

Let's Talk about Standardization — Starting with Frontend

It's common to demonstrate the TDD approach on tasks with well-defined logic that more closely resemble the backend's core logic: calculating a bowling match score, validating passwords, or calculating the total cost of a shopping cart in an online store. Such examples make it feel like TDD is something for instructional tasks. Thank you so much for the demonstration, but we have nowhere to apply it.

For Tourmaline Core, TDD, as bizarre as it may seem, started with the frontend and only then segued into other areas. Let's not look at the beginning of that journey: there, we're painfully comparing different cases and trying to figure out what our basic testing strategy will look like. Let's look at the present: what kind of workflow helps us develop quickly with tests these days.

Let's take an example. Imagine that a new isolated section appears on our frontend. It has a circle with a user's profile picture, their name, and the number of unread messages.



Now let's proceed step by step:

1 Establish a contract between the frontend and the backend.

This allows us to develop the frontend in our own sandbox, independently of the backend, and be sure that the system parts will fit together as expected in the end.

Confirm with the backend developer that the data for this section will be returned in line with this contract:

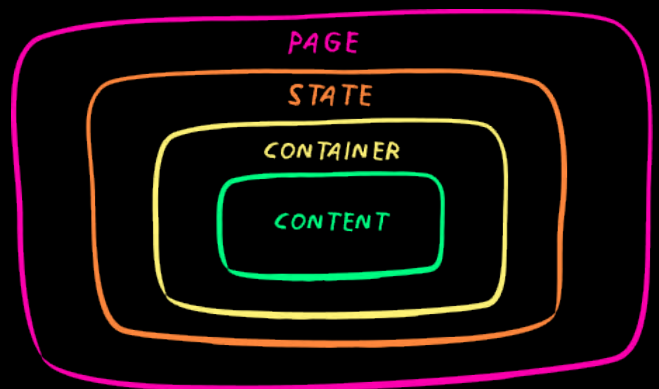
```
GET /user-info
{
  userPictureUrl: string,
  userFullName: string,
  numberOfUnreadMessages: number
}
```

2 Create a basic architecture design.

We think through the components' hierarchy and layers, and do a minimal code design before moving on to writing tests.

We split the page into four levels: Page orchestrates sections, State stores data and business logic, Container is responsible for network calls, Content handles rendering and interactions. Isolated sections have their own Container, State, and Content.

The layers don't overlap in their areas of responsibility. State doesn't know anything about the network, and Container doesn't render anything. This slicing is the same across every project — a developer from one project understands another project's structure without onboarding.



This section will look like this in the repository:

```
✓ features / layout / sections / user-info
  ✓ state
    TS UserInfoState.ts
    ⚙️ UserInfoStateContext.tsx
    ⚙️ UserInfoContainer.tsx
    ⚙️ UserInfoContent.tsx
```

3 Start with a basic scenario.

The most costly failure on the frontend is the one that prevents a user from going through the key business scenario. E2E tests that cover key use cases are the best at catching such failures.

Come up with a happy path use case.

At this point, it might sound like this: “I open the main page and see my name, profile picture, and the number of unread messages”. We don’t consider failure cases here — this will be a task for other tests.

Turn the use case into an E2E test.

Write the test using the library of your choice. Run the test and see that it’s red. That is correct — we haven’t added any implementation yet. We used the test to define and document the expected result.

Write the implementation.

We write code based on the architecture we’ve planned out. We write the code without considering any edge cases: as if error handling didn’t exist, network calls never failed because of timeouts, and the backend always returned a 200 status. We run the test. We make the test green: that will signify that our expectations matched reality and we reached the state we predicted.

4 Moving on to edge cases.

We ask ourselves: what could go wrong in the basic scenario?

Work on the Content layer.

What does the content look like with no unread messages? How do we render the UI when a user doesn’t have a profile picture, do we use a placeholder? For every question, we write a test that defines the expected behaviour, run it, make sure it fails at first, and then add the minimal implementation necessary for the test to pass.

Work on the State layer.

Let’s say a person’s name must be shortened on mobile devices: instead of John Doe — J. Doe. How will this logic work? Or, for example, the designer’s idea is that if the number of notifications exceeds 99, we shouldn’t render the full number and should use “99+” instead. We document all such cases with tests and add the necessary implementation.

Work on the Container layer.

What will happen if an error occurs while loading data? Do we show a loading indicator while loading data over the network? We come up with the answers while writing tests and add the required code.

You can watch a workshop on how to do all this in React here:



WORKSHOP
RECORDING



Alex, CEO of Tourmaline Core, speaking at DevOps

Conclusion

The layers give us hints on what we should take into consideration and make us document the expected behaviour for every question. When reviewing such code with tests and previously described use cases as guides, it becomes easier to figure out which situations were carefully thought through and which ones may have been overlooked. For many types of components, we can define test checklists at the company level: simple components should be implemented in the same manner by different developers.

Interface Exactly as Designed: Pixel-Perfect Layout with TDD

There are projects where the perfection of an interface is a big part of the overall impression of a premium product. For example, we had a situation where the client checked under the microscope (not a metaphor, actually using a microscope!) that the layout design was identical down to the pixel to what the developers did on the device's screen, with a gradient button. We polished that gradient to perfection.

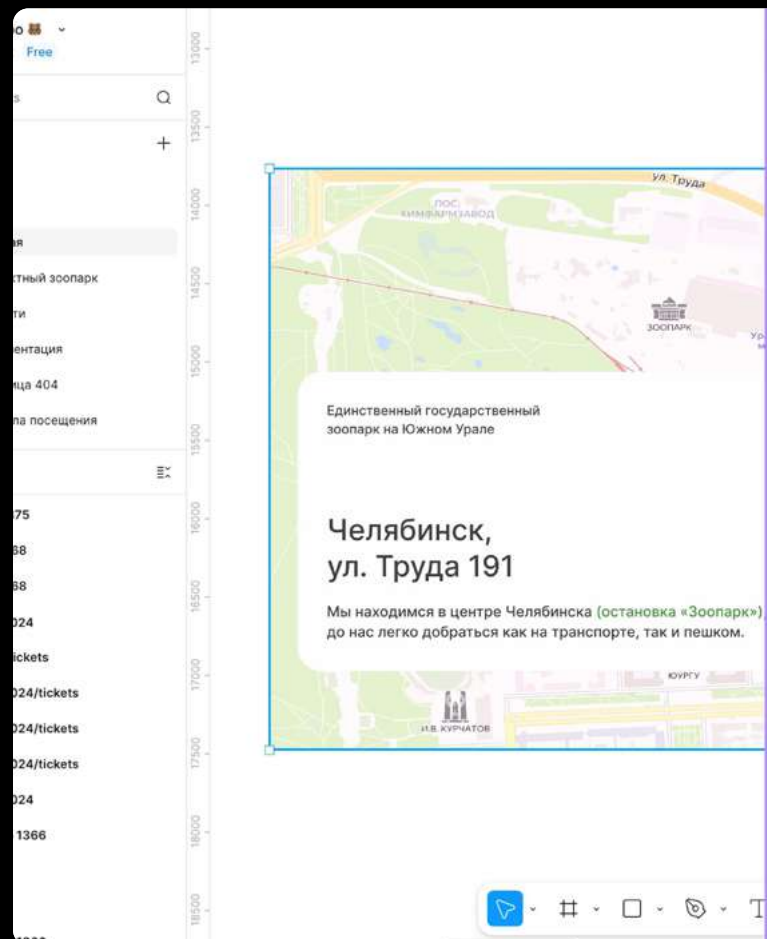
Another case when the layout should be carefully checked is a design system component.

A design system is shared across multiple interfaces: the same look of buttons, checkboxes, input fields all over. The task for a team of developers responsible for the design system is to implement it so it looks exactly the same as it does in the designers' mockups. If they don't, there's a risk that everything will look pretty in the designer's layout but will go sideways in development because of a couple of pixels or an element having a different colour.



What if we add a button in one place but the interface gets messed up in a completely different one because of that?

Previously, compliance with the design was checked manually: the layout design was compared to the website by eye. Then people started using a plugin when implementing web page layouts that could overlay a semi-transparent picture of the layout onto the web page. It became more convenient to check the interface, but it remained a manual process. Besides, interfaces don't only exist in the browser — they exist on ATM screens, mobile apps, TV, smartwatches: anywhere there's a display. It may not be possible to lay a layout picture over them with a plugin. And the more pages, tabs, pop-up windows a website has, the more time it takes to check them visually.



With the appearance of testing tools (for the web, these can be Cypress or Playwright, for example), screenshot testing can be automated and run after any layout change — a single two-minute pipeline, and you can rest assured that the interface is fine.

To do that, you only need two pictures: an example image from the designer and a screenshot of the implementation result. A tool overlays the pictures one on top of the other and compares each pixel. Misaligned pixels light up red. That means the layout isn't the exact same as the version approved by the designer anymore.



Screenshot testing mismatch



Chelyabinsk Zoo website

When we say we work using TDD, clients ask for examples. It's not possible with commercial projects due to NDAs. But we have an open-source project for a client — the website for the Chelyabinsk Zoo. Here we went all out: tested everything we could, layout implementation included.

The main question was:

HOW DO YOU
IMPLEMENT A LAYOUT
USING TDD?

The answer — through screenshot testing.

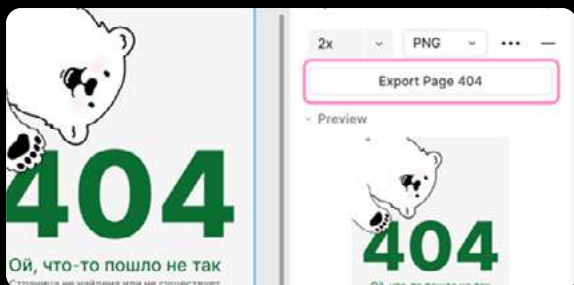


CASE STUDY
OF THE ZOO
WEBSITE

Here's how the layout can be implemented using TDD:

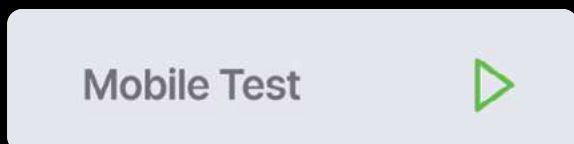
1 Export images of components or entire screens for every viewport size from Figma.

We usually create designs for mobile devices, tablets, desktops, and wide screens. These images serve as a specification for the layout.



2 Wrap images into automated tests.

To run automated comparisons of the screenshots against what the developers will implement.



3 Implement according to the layout.

Achieve the maximum similarity to the images from Figma.

4 Send it to the designer for approval.

A layout isn't always the same as its implementation. Sometimes there is a one-pixel difference between them that can't be fixed with styles. Then we show the difference to the designer and ask: "Is this OK?". After a confident "Yes!", we generate a new baseline image based on the implemented layout. Now this one will be used for screenshot testing. Until the approval, a test verifies compliance with the design; afterwards, it checks consistency of the results.

5 Run the test.

And see that it's green. It's no longer scary to change the project's layout — red pixels will tell us if something goes wrong.



Ekaterina and Artem, frontend developers at Tourmaline Core. Recording of the talk.



RECORDING
OF THE TALK
ON TDD FOR LAYOUT

The same test technique is applicable to mobile apps and other devices, the only thing that changes is the tool used for writing tests. The most important thing it should be able to do is overlaying an image on top of another and finding misaligned areas.

A Note on Refactoring

For some reason, when TDD is being demonstrated, the refactoring step is frequently skipped. Bada bing bada boom — and there it is, the polished code almost instantly. Makes it seem like no one ever writes drafts.

But of course, you can't go without the drafts in real life. Before starting a TDD cycle, we do some basic code design and bring the details to the final form as we go, partially with the help of tests.

And it may cause quite a despair when you find out during development that the architecture needs to be different: some part of the code needs to become a separate class, another needs to be removed from the method entirely. If tests are nailed to the implementation, they'll have to be rewritten as well. That fosters a feeling that TDD is just a hindrance that should be removed.

That's why tests should be kept at a higher layer — the behavioural one. Then you can change the draft all you want — the tests won't notice. There's an example further in this issue: on the backend, we replace an in-memory dictionary with a real database, change the internal flow of the endpoints entirely but leave the E2E test untouched.

Behind the Scenes of a Contract: Developing the Backend

Canonical examples don't help us to understand how to develop not only the frontend through tests but the backend as well: you take a look at how to test complicated functions, then open your backend and see the database calls and HTTP calls to other services. You're left confused: I should do... what, exactly?

Let's imagine we're writing the backend to work with users and we implement create-read-update-delete endpoints.

Here's what we're going to do:

1 Start with the contract between the frontend and the backend.

We agree on what the endpoints for data exchange will look like. Afterwards we declare the endpoints in the backend's code with all the request and response models but we don't add implementation at this point — a stub can be added instead. Based on that, the OpenAPI specification is generated, which in turn is used to generate a typed TypeScript client for the frontend. The frontend uses this client in its sandbox, while we're left with the list of everything that needs to be implemented.

2 Design the architecture.

We split the backend into layers following the onion architecture: API, Application, Core. The API is a thin layer. It takes a request, runs all the decorators (middlewares), for example, checks account permissions, and delegates request processing to the next layer.

The Application layer implements all scenarios: it works with the database, turns entities into response models, makes calls to the neighbouring services. Here is where the biggest chunk of the workload happens. The code inside the Application layer is sliced by features: everything related to user creation is in a single folder — the request, response, command, handler, and their corresponding tests.

The Core layer contains domain entities and performs business calculations.

General rule: a layer can't call outer layers, only inner ones. The layers don't overlap and don't perform the work of their neighbours.



Here's how it's going to look in the repository once we're finished (using a .NET API as an example):

```

  ▾ Api/Features/Users/
    • UsersController.cs
    • UsersControllerTests.cs
  ▾ Application/Features/Users/Handlers/
    CreateUser/
    • CreateUserCommand.cs
    • CreateUserCommandTests.cs
    • CreateUserHandler.cs
    • CreateUserHandlerTests.cs
    • CreateUserResponse.cs
    • CreateUserRequest.cs
  ▾ Core/Features/Users/Entities
    • User.cs
    • UserTests.cs
```

We put tests next to the code without creating a separate mirroring folder with a duplicate structure: once upon a time, we were bitten by the frontend; since then, we've found it more convenient to have test files next to the implementation.

3 Start with a happy path scenario.

We draft the implementation code based on the happy path — when everything goes according to plan and nothing extraordinary happens.

Come up with a use case.

We end up with something like: “I create a user, read their info, update their name, and delete them”.

Write an E2E test for the happy path.

Basic rule: every endpoint needs to be called in an E2E test at least once. We create a user via an endpoint, get the list of all users and check that the newly created one is in that list; update their name, request that particular user and see that their name was updated; then we delete that user and verify that they are no longer in the list.

The cases where something goes wrong and exceptions occur are not these tests’ responsibility. E2E tests are responsible for everything good and high-level in this world. We write them using Karate and its declarative Gherkin language. These tests don’t depend on the backend’s language: they look the same for .NET, Python, and C++ backends. We run the test — it’s red because there is still a stub in the implementation. With the test, we’ve documented the expected result.

Write the implementation, distribute the code across the layers.

And there’s a special trick: to make an E2E test pass, you don’t need a database yet. The first implementation can store users in an in-memory dictionary. Martin Fowler calls that a fake — a functioning implementation that cuts corners and isn’t suitable for production. We code without taking edge cases into account, and write the minimum amount of code: our task is just to make the E2E happy path test pass.

Replace the fake with the actual database.

This is where entities, ORM (Object-Relational Mapping) operations, migrations, commands, and queries come in. We’ve changed the internal flow of the endpoints entirely — without touching a line in the E2E test. The test is still green as it checks the behaviour, not the implementation. That’s the power of E2E — these tests couldn’t care less about what tools were used for the implementation and how many layers it has.

We choose only happy-path E2Es to run in two cases: in a local environment, before merging changes into master, and in a production environment — regularly, following a schedule. This helps us verify that the key user scenarios are still up and running — that’s one of the pillars supporting an application’s observability.

4 Work through edge cases and distribute tests according to the architecture.

Core — tests almost without mocks.

The Core layer mostly contains domain logic, and usually consists of pure functions. Let’s say we put a surname, name, and a middle name in one line in a Core function. Here we can ask such questions as: what happens if there’s no middle name? And what if a surname consists of two words with a hyphen between them? Every question turns into a test with an input and an output, with no database and no HTTP calls.

Queries and commands — tests with a real database.

This isn’t about C# logic; it’s about what happens in SQL: filtering, sorting, grouping — and checking that against in-memory lists is useless. We start a database in Docker, add three users to it, and check that we get the expected two when applying particular filters. Every test runs in its own transaction with the Serializable isolation level, and every transaction rolls back after its test, so tests don’t interfere with each other. Another case where these tests are needed is when some of the business rules are enforced by the database: for example, the uniqueness of an email is ensured by a corresponding index.

Handlers — tests for the whole scenario.

Handlers are responsible for orchestrating the work with commands, queries, and external services. Handlers are also the very part where API response mapping happens. Here we test the behaviour, the branching and the cases where something goes wrong, for example, an external service crashes.

Controllers — an integration test via HTTP.

We start the application and send a request to an endpoint. We aren’t validating the business logic, we are validating the contract: with an empty required field, we expect to get a 400 status and see the error in its expected format in the response body. This layer may require mocks to emulate responses from other services, you may even disable working with the database if needed.

Coverage is calculated separately for every type of test: unit, integration, and E2E. There isn't just one coverage badge in the README, there are four of them. The difference between them tells you more than an overall number: we don't just see how much of the code is covered, we see what it's covered by; plus, that makes it easier to understand what the overlap between test layers is.



VIDEO SERIES
ABOUT TDD
ON BACKEND

When we start with a happy-path E2E test and then move on to detailing the edge cases on different layers — that's part of the iterative development approach. It can be compared to drawing a portrait. You can work incrementally: for example, draw eyes in detail, then move on to eyebrows and do them thoroughly as well. Or you can move iteratively: start by drafting a full portrait of a person, then gradually add the details everywhere little by little, slowly revealing the complete picture. It's the same with tests and implementing a new feature: we sketch out a general composition with a little bit of everything using an E2E test and a happy-path implementation; then we gradually start working out edge cases on different layers.

With such a TDD cycle, our picture emerges rhythmically.

Stitching the Hardware: Adding TDD to Embedded Development

TDD in embedded development seems to be a complicated subject

Since this approach means you'll have to write tests frequently and modify the implementation repeatedly, it also means you'll often have to rebuild the firmware and load it onto the device to run tests. For some long-lived systems with long development cycles, a full build, device flashing, and waiting for the test report take over ten minutes. As a result, running the tests takes so long that using TDD turns into a bad idea.

Testing the code only on a local machine is a poor choice: some problems may reveal themselves only on the device's platform if its architecture differs from the developer's host machine (which is almost always the case). A char can be signed or unsigned depending on the architecture and the compiler. A pointer is 8 bytes on the host and 4 bytes on a 32-bit target. A stack may be megabytes in size on the host and kilobytes on the target. Bam! And you have an out-of-nowhere bug that can't be reproduced locally.

So can you speed up the TDD cycle in embedded development?

You can try it using the **Dual Targeting technique**. The idea is that the same code gets compiled for two target platforms: a host and a target. A host is the developer's local machine, while a target is the platform the target device runs on. So we have the same codebase and two sets of toolchains and builds.

The architecture is built around three levels:

1 Logic — finite-state machines, calculations, schedulers.

It doesn't depend on the device, so it can be tested locally and go through final verification on the device. This level can be developed using TDD on the host.

2 Ports — narrow interfaces.

“Send a byte”, “set a timer”, “read memory”. On the host, we add a fake implementation that will help isolate development of the logic from the device, while on the target, we use the real drivers.

3 Drivers — a thin layer that connects the ports to the hardware.

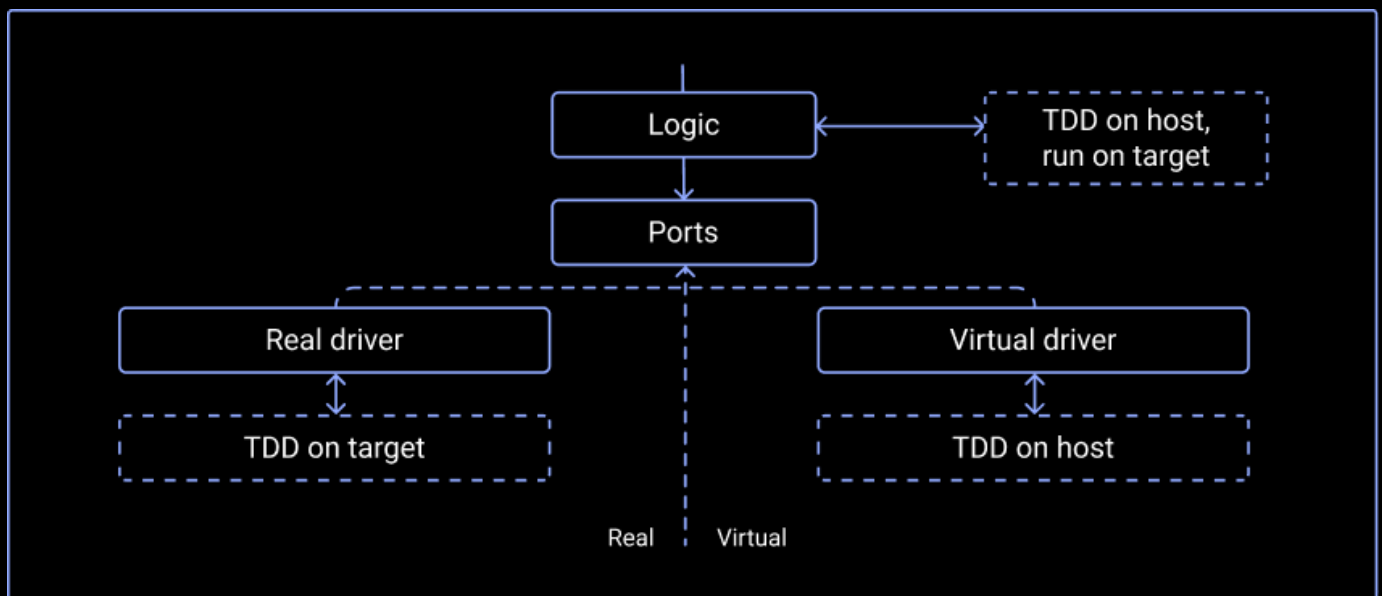
They're tested on the target, so they require a build and flashing the device. Working with TDD at this level remains difficult.

The logic from the upper level shouldn't include the header file to communicate with the platform; otherwise, the magic won't work: once again, you've tied the entire code to the hardware, and testing on the device becomes unavoidable. Faking the implementation is achieved by separating the interface from the implementation: one interface, two implementations, the required one is chosen based on the build target. No `#ifdef` in the logic.

The proportion of code at each level depends on the domain: a device that triggers a relay in response to an external signal has barely any logic, so Dual Targeting wouldn't play a big part. Meanwhile, a device with calculations, data updates over the network, state processing, and complex algorithms may have a large proportion of logic. That means most of the development can go through a quick TDD cycle on the host machine.

In this case, we run tests locally on every TDD cycle, while builds for the target platform and tests on hardware only happen on pull requests to the master branch.

Disclaimer: race conditions with interruptions, real deadlocks and stack depth issues won't be caught by the tests on the host machine. Dual Targeting removes unnecessary waiting from the TDD cycle, but not the necessity to test on the target.



Bonus: sanitizers

ASan and TSan can't be run on the hardware, while they can be enabled on the host with a single flag. The same caveat as everywhere else: they check the host build, not the target one.

TDD means dozens of cycles an hour. Without Dual Targeting, 40 iterations at 10 minutes each take almost a full workday. On the host, the same 40 can fit in an hour. That's why, in embedded development, the question isn't really "TDD or hardware?". You can do it this way: a speedy TDD cycle — on the host, one final check — on the hardware.

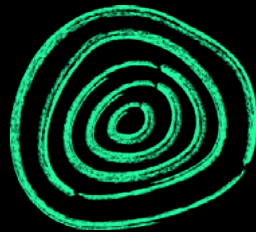
Where Do QA Engineers Live in the TDD World?

Tactical responsibility for quality and testing rests on the shoulders of every developer on the team. Strategic one — on the shoulders of a QA engineer

With this approach, QA engineers don't just run manual tests or write automated tests for existing code. They resemble a player-coach who plans out the testing strategy with the team, keeps an eye on its execution and teaches developers which tests to write at which level.

A QA engineer is a coach who thinks through what testing system the project will have and helps in non-trivial situations when it's unclear where a particular test belongs.

On paper, a testing strategy often resembles a perfect slice of an onion. Every layer is neatly placed and doesn't overlap the others. This is an onion that we see in an ad for the tastiest burger or at a vegetable competition:



ONION IN AN AD



ONION AT HOME

It's great if your QA engineer has come up with a strategy that covers most potential questions, and in a tough situation every developer turns to the strategy and finds an answer to their question right away: yes, your tests for CRUD scenarios should look like this. In a perfect world, this chosen testing approach also comes with an explanation and a list of alternatives that were considered but got rejected.

When it comes to a real-life onion, a QA engineer has to help figure out how to deal with the situation gracefully and how to integrate it into the overall testing pattern in a different but sensible way.

This includes coming up with ideas on how to standardize tests, how to act in uncommon cases, thinking ahead about what issues may come up next, checking the tests written by developers, and being the backbone for the team and the product owner. Lighting the way and making it a concrete bridge, not a shaky rope one, is also part of the job. So is regularly verifying that this bridge is doing fine, not cracking at the seams, begging for renovation.

All in all, a QA engineer's task is to systematically monitor the software's quality.

The Age of AI — Quality Standards Are Higher Than Ever

AI stormed into our lives, speeding up the code-writing process. And that's exactly the moment when checking code quality becomes the most important thing.

Here TDD is in a particularly strong position. A test written before code is basically a task specification:

"HERE'S AN INPUT, HERE'S AN EXPECTED OUTPUT, HERE ARE EDGE CASES"

The TDD cycle when working with AI looks like this: an engineer writes a test and documents the behaviour; a model generates the implementation; the test validates the result. If the test fails, the model sees the specific difference between the expected and actual result. At the same time, the accumulated capital of automated tests ensures that the change hasn't broken other parts of the system.

Isn't that the renaissance of Test-Driven Development?

The emergence of AI served as a catalyst for improving structure and architecture: it is now more crucial than ever to have specifications, contracts, user stories, an iterative development approach, and well-written automated tests that help ensure the quality of the developed product. What was considered overhead for years is now becoming a set of approaches to make AI useful and effective.

AI created a new argument in favor of TDD: developing through tests is a means to get predictable rather than merely plausible code from AI. Because quick code generation also means quick accumulation of technical debt. And technical debt is known to be a business problem.

A team that has a testing strategy and a habit of practicing TDD gets better output from AI than a team that doesn't.



ABOUT TECHNICAL
DEBT

Passing the Mic: What Our Team Thinks



Alex

CEO, integrated TDD into processes



Anton

team lead of embedded development, worked with hardware using TDD



Anastasia

full-stack developer, three years of TDD in web development



Snezhana

ML/backend engineer, now transitioning from tests after code to TDD

How much did the transition to TDD cost you?

Alex *CEO*

How many tears? Preparation for the transition may take a while as you gradually try to add testing to the development process. It took three years for one team of five engineers. At one point, the team themselves said: "We will work using TDD". There was no need for further convincing or for suggesting an experiment. TDD is hard to integrate from the top down, but it's possible. You can say: "From now on, we'll be working this way". But if you aren't helping the process, it won't work. Changes are already hard to introduce since there's always resistance. All in all, the transition cost five tears, three years, and one huge stroke of luck to have such colleagues brought into my life by fate. Convert that into money yourself.

Anton *Team lead of embedded development*

X hours of the team's time to build a habit and reshape the code-writing approach, plus a gazillion extra cups of coffee and 10ml of the team lead's tears.

Anastasia *Full-stack developer*

My "currencies" were time and nerves, along with a long battle against the reluctance to write tests before code instead of rushing to implement features as fast as possible (despite understanding perfectly well how much it would simplify and protect the development process). I was also constantly confused about how to write a test before you have a full picture of the implementation.

Snezhana *ML/backend engineer*

For me, the transition process is far from finished, but based on my experience on a recent project — I'm at net zero in terms of time. Meaning, the extra time spent in the beginning on coming up with test scenarios was offset by the time saved during the implementation because it was already clear what functionality should be included.

What argument for TDD do people keep repeating though it doesn't work?

Alex *CEO*

TDD gives 100% test coverage. I'd question that — it's a myth. Coverage isn't the goal. It may happen, but on real projects, when you apply TDD and don't turn it into a cargo cult, the coverage may be lower because you may decide not to test something at all. TDD doesn't exist in isolation but in connection with the testing strategy. And if full coverage is not part of the strategy, it won't happen.



Anton *Team lead of embedded development*

Time savings. There seem to be some time savings in the long run, but at first glance, it doesn't feel like there is any at the beginning of the development. The team is often sceptical as well, and pushes for a "here and now" solution.



Anastasia *Full-stack developer*

Green tests give the feeling that all the cases are covered, but in actuality, it's a feeling of completeness, not completeness itself. The list of cases is only as good as it was put together, and TDD doesn't add anything to that list by itself.



Snezhana *ML/backend engineer*

Probably, it's that TDD makes you think ahead, therefore decreases the number of bugs later. This argument may work but it's important to note that it doesn't apply to every project. For me, it often didn't work because many of the tasks were research-focused without a clear specification of the expected result. Designing tests in advance doesn't work when you can only define the rough direction of an R&D task.



What annoys you personally about TDD?

Alex *CEO*

What annoys me is that when I follow my own interpretation and take a pragmatic approach rather than canonical TDD, and don't try to call it something different, like ATDD, the professional community sometimes doesn't recognize it as TDD, or I have to justify my approach.



Anton *Team lead of embedded development*

TDD slams on the brakes and doesn't let you write code from the jump. Imagine you already have a genius solution in your head, all that's left to do is create a new function and add five lines of code to it. But no, it turns out that the test asks several unpleasant questions that we didn't consider at first.



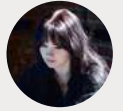


Anastasia *Full-stack developer*

A slowdown in development at the start; always having to think about tests when changing code.

Snezhana *ML/backend engineer*

When you feel like you've thought everything through — you still haven't... You can finalize the list of tests, merge, and then a new change is requested that requires adding more tests. And you have to resist the urge to skip the testing step and implement that change right away. So it's one thing to develop with TDD at the beginning of a project, and another, a more annoying one — to stick to TDD cycles in the future, especially for smaller features.



Where didn't TDD help you?



Alex *CEO*

TDD by itself doesn't help develop a team's discipline. TDD by itself doesn't make a team highly proficient in its tech stack or in testing. But TDD can act as a catalyst for whatever a team lacks: responsibility, engagement, interest in personal and team growth — in engineering culture, discipline, and tech expertise.

Anton *Team lead of embedded development*

Hardware-dependent code. Sometimes hardware decides that it's "smarter" than code.



Anastasia *Full-stack developer*

A bug that's barely reproducible because it's tied to external processes and the environment. Writing a test to reproduce such a bug before fixing it is almost impossible, and the "red test first" rule becomes absurd.

Snezhana *ML/backend engineer*

It's not that it "didn't help" necessarily; it was just unclear how to use it. This happens with R&D projects, where there is a concept of the desired result but the path to it is unclear. What exactly do we test? How is it going to work? What are we developing in general? At the stage of testing out hypotheses, it's irrational to write a test for a function that may become useless tomorrow. We also couldn't use TDD when developing a chat bot: we didn't know how to test bots, while the deadlines were tight. TDD takes more time at the initial stages as it is, and if you aren't familiar with how to test what you're developing — even more so.



At what point did you believe in TDD?

Alex *CEO*

Believed? I'm not that gullible. I tried it and saw how it worked. My leap from "tests after code" to "tests before code" happened when I listened to a colleague's talk on the value of TDD for product quality and engineering culture. Back then, he said: "As engineers, we don't have an excuse not to use TDD". I thought: "Well, yes, there's nothing preventing us from doing it. All that's left is to start working this way as a principle".

But I really believed in it after introducing it to the company's teams, when I didn't see a single bug in production during the three months of developing the time tracker. Everything was stable without a QA engineer on the team.



Anton *Team lead of embedded development*

When the client sent another round of edits, we had to change the logic of several modules. We changed the implementation, ran the tests — and saw right away what broke and what continued to work as before. We're moving confidently, taking small steps.



Anastasia *Full-stack developer*

I believed in the idea from the start. And I believe in it a little more every time, especially during refactoring, when all the tests stay green. Despite the initial difficulties, TDD gives confidence that the developed system works as intended.



Snezhana *ML/backend engineer*

The first thing that comes to mind is a situation we encountered during development, when we had to use an uncommon solution that barely had any disadvantages except for one: an error could only be detected when a particular endpoint was called. We could go with that solution because we were sure: all endpoints get called when E2E tests are run, so an error won't go undetected in any case and won't leak to prod.



How to start working with TDD?

We know how hard it is to transition a team to TDD after many years of doing things differently. But it seems that now is the best time to try it.

You can start small: by writing tests for bugs as you find them. Once you find a bug, document it with an automated test and only then fix it. After a month of such work, the team will have a set of regression tests and a habit of starting with a test. That will be an easy way in.

And may Martin Fowler and his quote help you:

"No bug is considered properly fixed without an automated regression test. In a unit testing culture, when a bug is discovered, the natural reaction is to write a test that exposes it, then to fix the code to squash it".

And here's also Kent Beck to help:

"Test-driven development is a way of managing fear during programming... And tests give a developer the courage to change, remove, and add".



You can also reach out to us to talk about TDD: we'll be happy to share our experience.

MAP OF OUR PUBLIC RESOURCES ON THE TOPIC



This magazine is only a summary, while behind the QR code are all the resources we've accumulated over our seven-year TDD journey. Conference talks, workshops, a mini-course on the .NET-backend using TDD, and open-source repositories of the projects we've talked about.



SHARE YOUR THOUGHTS
OR LEAVE A REVIEW



LEARN MORE ABOUT
THE COMPANY